

Tabelas *Hash*

Unidade II: Tipos Abstratos de Dados

Tabela *Hash*

- Estrutura de dados em que acessamos elementos com $\Theta(1)$
- Tem um *array* e uma função de transformação sobre a chave de pesquisa

Vantagens

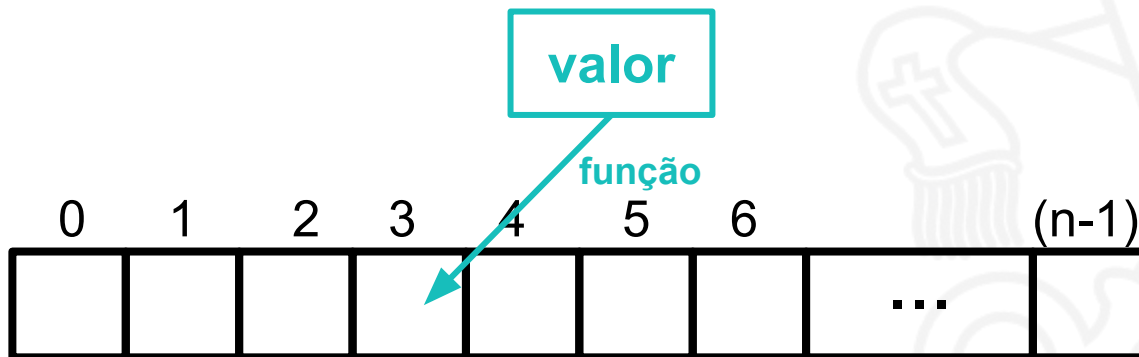
- Eficiência no custo de pesquisa: $\Theta(1)$ para o caso médio
- Simplicidade de implementação

Desvantagens

- Custo elevado para recuperar os registros de forma ordenada, sendo necessário ordenar o arquivo
- Pior caso é $\Theta(n)$

Função de Transformação

- Recebe um valor e retorna uma chave (ou posição) do valor em um *array* (ou tabela)
- Definida pelo projetista do sistema



Exemplo do Aniversário

- Temos uma tabela com 366 entradas (dias) na qual inserimos pessoas conforme suas datas de nascimento

- 1º de janeiro $\Rightarrow$ posição 0
- 2 de janeiro $\Rightarrow$ posição 1
- 3 de janeiro $\Rightarrow$ posição 2
- 4 de janeiro $\Rightarrow$ posição 3
- 5 de janeiro $\Rightarrow$ posição 4 ($0 + 5 - 1$)
- 13 de março $\Rightarrow$ posição 72 ($31 + 29$ + $13 - 1$)
- 26 de maio $\Rightarrow$ posição 146 ($31 + 29 + 31 + 30$ + $26 - 1$)
- 13 de setembro $\Rightarrow$ posição 256 ($31 + 29 + 31 + 30 + 31 + 30 + 31 + 31$ + $13 - 1$)

Aniversário	Pessoa
5/1	Paulo
13/3	Lica
26/5	Olímpia
13/9	João

Exercício Resolvido (1)

- Faça um programa contendo uma *Hashtable* que armazena pessoas (classe contendo os atributos nome e data de aniversário) e mostra as seguintes opções de escolha para o usuário: 1 - inserir pessoa; 2 - remover pessoa; 3 - listar pessoas; 4 - pesquisar pessoa; e 5 - sair. Quando o usuário escolher uma das opções, o programa realiza a operação escolhida na Hashtable. Use a função de transformação proposta no exemplo do aniversário.

Exercício Resolvido (1)

- Faça um programa contendo uma *Hashtable* que armazena pessoas (classe contendo os atributos nome e data de aniversário) e mostra as seguintes opções de escolha para o usuário: 1 - inserir pessoa; 2 - remover pessoa; 3 - listar pessoas; 4 - pesquisar pessoa; e 5 - sair. Quando o usuário escolhe uma opção, o programa realiza a operação escolhida e mostra o resultado de transformação proposta no exemplo.

Pause!

Exercício Resolvido (1)

```
class Pessoa {
    public string nome;
    public DateTime dataNas;
    public Pessoa() {
        this.nome = "";
        this.dataNas = DateTime.Today;
    }
    public Pessoa(string nome, int anoNas, int mesNas,
int diaNas) {
        this.nome = nome;
        this.dataNas = new DateTime(anoNas, mesNas, diaNas);
    }
    public void Mostrar() {
        Console.WriteLine(nome+" (" +dataNas.ToString()+" )");
    }
    public int hash() {
        return hash(dataNas.Month, dataNas.Day);
    }
}
```

```
public static int hash(int mesNas, int diaNas) {
    int resp;
    if (mesNas == 1) { resp = diaNas - 1;
    } else if (mesNas == 2) { resp = 31 + diaNas - 1;
    } else if (mesNas == 3) { resp = 60 + diaNas - 1;
    } else if (mesNas == 4) { resp = 91 + diaNas - 1;
    } else if (mesNas == 5) { resp = 121 + diaNas - 1;
    } else if (mesNas == 6) { resp = 152 + diaNas - 1;
    } else if (mesNas == 7) { resp = 182 + diaNas - 1;
    } else if (mesNas == 8) { resp = 213 + diaNas - 1;
    } else if (mesNas == 9) { resp = 244 + diaNas - 1;
    } else if (mesNas == 10) { resp = 274 + diaNas - 1;
    } else if (mesNas == 11) { resp = 305 + diaNas - 1;
    } else { resp = 335 + diaNas - 1;
    }
    return resp;
}}
```

Exercício Resolvido (1)

```
class ExercicioResolvido01TabelaHash {
    public static void Main (string[] args) {
        Hashtable th = new Hashtable();
        int opcao;
        do {
            Console.WriteLine("Digite uma opção:");
            Console.WriteLine("1 - inserir elemento");
            Console.WriteLine("2 - remover elemento");
            Console.WriteLine("3 - listar elementos");
            Console.WriteLine("4 - pesquisar elemento");
            Console.WriteLine("5 - sair");
            opcao = int.Parse(Console.ReadLine());
            if (opcao == 1){
            }else if (opcao == 2){
            }else if (opcao == 3){
            }else if (opcao == 4){
            }else if (opcao == 5){
            }else{
            }
        } while (opcao != 5);    } }
```

mazena pessoas
(sário) e mostra as
erir pessoa; 2 -
essoa; e 5 - sair.
rama realiza a
transformação

Exercício Resolvido (1)

```
class ExercicioComplementarU01S02EC03 {  
    public static void Main (string[] args) {  
        Hashtable th = new Hashtable();  
        int opcao;  
        do {  
            Console.WriteLine("1 - Inserir pessoa na agenda; 2 - Excluir pessoa da agenda; 3 - Pesquisar pessoa; 4 - Listar agenda; 5 - sair.");  
            Console.WriteLine("Digite a opção: ");  
            Console.WriteLine("Ano de nascimento da pessoa a ser inserida: ");  
            int anoNas = int.Parse(Console.ReadLine());  
            Console.WriteLine("Mês de nascimento da pessoa a ser inserida: ");  
            int mesNas = int.Parse(Console.ReadLine());  
            Console.WriteLine("Dia de nascimento da pessoa a ser inserida: ");  
            int diaNas = int.Parse(Console.ReadLine());  
            pessoa.dataNas = new DateTime(anoNas, mesNas, diaNas);  
            th.Add(pessoa.hash(), pessoa);  
        }  
        else if (opcao == 5){  
            Console.WriteLine("Programa encerrado.");  
        }  
        else{  
            Console.WriteLine("Opção inválida. Digite uma opção válida.");  
        }  
    } while (opcao != 5);  
}
```

armazena pessoas
(sábado) e mostra as
a; 2 -
- sair.
za a
ação

Exercício Resolvido (1)

```
class ExercicioComplementarU01S02EC03 {  
    public static void Main (string[] args) {  
        Hashtable th = new Hashtable();  
        int opcao;  
        do {  
            if (opcao == 2) {  
                Console.WriteLine("Mês de nascimento da pessoa a ser removida: ");  
                int mesNas = int.Parse(Console.ReadLine());  
                Console.WriteLine("Dia de nascimento da pessoa a ser removida: ");  
                int diaNas = int.Parse(Console.ReadLine());  
                th.Remove(Pessoa.hash(mesNas, diaNas));  
                Console.WriteLine("Removido com sucesso!");  
            }  
            Console.WriteLine("5 - sair");  
            opcao = int.Parse(Console.ReadLine());  
            if (opcao == 1) {  
            } else if (opcao == 2) {  
            } else if (opcao == 3) {  
            } else if (opcao == 4) {  
            } else if (opcao == 5) {  
            } else {  
            }  
        } while (opcao != 5);  
    }  
}
```

mazena pessoas
(sário) e mostra as
a; 2 -
- sair.
za a
ação

Exercício Resolvido (1)

```
class ExercicioComplementarU01S02EC03 {  
    public static void Main (string[] args) {  
        Hashtable th = new Hashtable();  
        int opcao;  
        do {  
            Console.WriteLine("1 - Adicionar pessoa;  
2 - Remover pessoa;  
3 - Mostrar pessoas;  
4 - Salvar arquivo;  
5 - Sair.");  
            Console.WriteLine("Digite a opção:");  
            opcao = int.Parse(Console.ReadLine());  
            if (opcao == 1){  
                Console.WriteLine("Adicionar pessoa");  
            }  
            else if (opcao == 2){  
                Console.WriteLine("Remover pessoa");  
            }  
            else if (opcao == 3){  
                Console.WriteLine("Mostrar Hashtable:");  
                foreach (DictionaryEntry pair in th){  
                    Console.WriteLine(((Pessoa)pair.Value).Mostrar());  
                }  
                Console.WriteLine();  
            }  
            else if (opcao == 4){  
                Console.WriteLine("Salvar arquivo");  
            }  
            else if (opcao == 5){  
                Console.WriteLine("Sair");  
            }  
            else{  
                Console.WriteLine("Opção inválida");  
            }  
        } while (opcao != 5);  
    }  
}
```

armazenar pessoas
(usuário) e mostra as
opções; 2 -
3 - sair.
4 - Salvar a
arquivo

Exercício Resolvido (1)

```
class ExercicioComplementarU01S02EC03 {
    public static void Main (string[] args) {
        Hashtable th = new Hashtable();
        int opcao;
        do {
            if (opcao == 4){
                Console.WriteLine("Mês de nascimento da pessoa a ser pesquisada: ");
                int mesNas = int.Parse(Console.ReadLine());
                Console.WriteLine("Dia de nascimento da pessoa a ser pesquisada: ");
                int diaNas = int.Parse(Console.ReadLine());
                bool contains = th.Contains(Pessoa.hash(mesNas,diaNas));
                Console.WriteLine("Contains({0}/{1}): {2}", diaNas, mesNas, contains);
            }
            opcao = int.Parse(Console.ReadLine());
        } while (opcao != 5);
    }
}
```

mazena pessoas
(sário) e mostra as
a; 2 -
- sair.
za a
ação

Exercício Resolvido (1)

```
class ExercicioComplementarU01S02EC03 {  
    public static void Main (string[] args) {  
        Hashtable ht = new Hashtable();  
        int opcao;  
        do {  
            if (opcao == 5){  
                Console.WriteLine("FIM DE PROGRAMA");  
            } else {  
                Console.WriteLine("Opção inválida. Por favor, tente novamente.");  
            }  
            Console.WriteLine("3 - listar elementos");  
            Console.WriteLine("4 - pesquisar elemento");  
            Console.WriteLine("5 - sair");  
            opcao = int.Parse(Console.ReadLine());  
            if (opcao == 1){  
            }else if (opcao == 2){  
            }else if (opcao == 3){  
            }else if (opcao == 4){  
            }else if (opcao == 5){  
            }else{  
            }  
        } while (opcao != 5);  
    }  
}
```

mazena pessoas
(sário) e mostra as
a; 2 -
- sair.
za a
transformação

Algumas Considerações

- Normalmente, a função de transformação **não** depende do número de itens da coleção
- O tamanho da tabela deve ser escolhido com atenção, por exemplo, ele pode ser um número primo
- As chaves não numéricas devem ser transformadas em números

Outro Exemplo de Função de Transformação

- $h(k) = k \% m$, onde k é o somatório do código ASCII de todos os caracteres e m , o tamanho da tabela. Por exemplo, sendo $m = 13$, temos:

$$h(\text{oscar}) = 536 \% 13 = 3$$

$$h(\text{bosco}) = 534 \% 13 = 1$$

$$h(\text{elisa}) = 526 \% 13 = 6$$

$$h(\text{maya}) = 424 \% 13 = 8$$

$$h(\text{lucio}) = 540 \% 13 = 7$$

$$h(\text{paulo}) = 545 \% 13 = 12$$

$$h(\text{sofia}) = 530 \% 13 = 10$$

$$h(\text{sergio}) = 649 \% 13 = 12$$

Outro Exemplo de Função de Transformação

- Inserimos, pesquisamos ou removemos um item a partir de sua chave:
 - $h(\text{maya}) = 8$
 - $h(\text{ana}) = 5$
 - $h(\text{sergio}) = 12$
 - **Como podemos tratar das colisões?**

0	
1	bosco
2	
3	oscar
4	
5	
6	elisa
7	lucio
8	maya
9	
10	sofia
11	
12	paulo

Colisões Primárias

- Acontecem quando inserimos um elemento em uma posição do *array* ocupada por outro elemento (mesma chave)
- Na verdade, a função de transformação dá acesso a um subconjunto de elementos
- **Solução:** Inserir o elemento com colisão em outra posição da tabela ou em uma estrutura externa

Gerenciamento de Colisões

- **Métodos de transformação direta (resolução por cálculo):** calculam uma nova posição baseada na chave
- **Métodos de transformação indireta (resolução por encadeamento)** encadeiam os elementos com mesma chave

Gerenciamento de Colisões

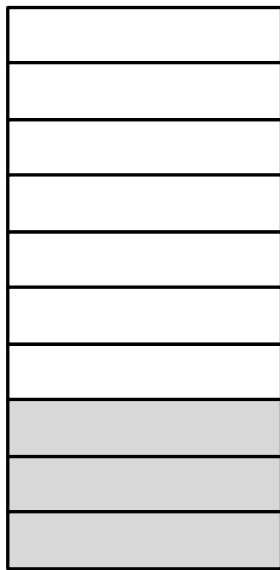
- *Hash* direta com área de reserva (*overflow*)
- *Hash* direta com *rehash*
- *Hash* indireta com lista flexível simples

Gerenciamento de Colisões

- *Hash* direta com área de reserva (*overflow*)
- *Hash* direta com *rehash*
- *Hash* indireta com lista flexível simples

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva



Tamanho da tabela

Tamanho da área de reserva

```
int hash (int x){  
    return x % tamTabela;  
}
```

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva


```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	
1	
2	
3	
4	
livre 5	
6	
7	
8	
9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:

5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	
1	
2	
3	
4	
livre 5	5
6	
7	
8	
9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:

5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	
1	
2	
3	
4	
5	5
6	
7	
8	
9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

livre	0	
	1	
	2	
	3	
	4	
	5	5
	6	
	7	
	8	
	9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:

5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

livre	0	7
	1	
	2	
	3	
	4	
	5	5
	6	
	7	
	8	
	9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	
8	
9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7	ocupado
1		
2		
3		
4		
5	5	
6		
7		
8		
9		

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
livre 7	
8	
9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	
9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7	ocupado
1		
2		
3		
4		
5	5	
6		
7	14	
8		
9		

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
livre 8	
9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
9	

ocupado

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
livre 9	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Hash Direta com Área de Reserva (overflow)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
9	19

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21 e 19

Exercício Resolvido (2)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
9	19

```
int hash (int x){  
    return x % 7;  
}
```

Exercício: Quantas comparações são necessárias para pesquisar o:

- a) 5 b) 21
c) 19 d) 6

Exercício Resolvido (2)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	
6	
7	14
8	21
9	19

```
int hash (int x){  
    return x % 7;  
}
```

Pause!

as comparações
para pesquisar o:

- a) 5 b) 21
c) 19 d) 6

Exercício Resolvido (2)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
9	19

```
int hash (int x){  
    return x % 7;  
}
```

Exercício: Quantas comparações são necessárias para pesquisar o:

- a) 5 1 cmp b) 21
c) 19 d) 6

Exercício Resolvido (2)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
9	19

```
int hash (int x){  
    return x % 7;  
}
```

Exercício: Quantas comparações são necessárias para pesquisar o:

- a) 5 b) 21 3 cmp
c) 19 d) 6

Exercício Resolvido (2)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
9	19

```
int hash (int x){  
    return x % 7;  
}
```

Exercício: Quantas comparações são necessárias para pesquisar o:

- a) 5 b) 21
c) 19 4 cmp d) 6

Exercício Resolvido (2)

- Suponha uma tabela de tamanho 7 e com 3 na área de reserva

0	7
1	
2	
3	
4	
5	5
6	
7	14
8	21
9	19

```
int hash (int x){  
    return x % 7;  
}
```

Exercício: Quantas comparações são necessárias para pesquisar o:

a) 5

b) 21

c) 19

d) 6

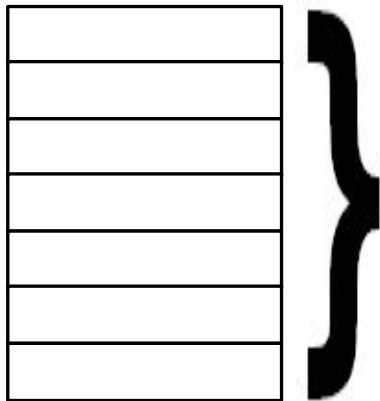
1 cmp

Gerenciamento de Colisões

- *Hash* direta com área de reserva (*overflow*)
- *Hash* direta com *rehash*
- *Hash* indireta com lista flexível simples

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7



Tamanho da tabela

```
int hash (int x){  
    return x % tamTabela;  
}
```

```
int rehash (int x){  
    return ++x % tamTabela;  
}
```

Hash Direta com *Rehash*

- Suponha uma tabela de tamanho 7

0	
1	
2	
3	
4	
5	
6	

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

livre

0	
1	
2	
3	
4	
5	
6	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:

5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

livre

0	
1	
2	
3	
4	
5	5
6	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:

5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	
1	
2	
3	
4	
5	5
6	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

livre

0	
1	
2	
3	
4	
5	5
6	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

livre

0	7
1	
2	
3	
4	
5	5
6	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	
2	
3	
4	
5	5
6	

ocupado

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

livre

0	7
1	
2	
3	
4	
5	5
6	

```
int rehash (int x){  
    return ++x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	

```
int rehash (int x){  
    return ++x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com *Rehash*

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	

ocupado

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	

ocupado

```
int rehash (int x){  
    return ++x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	

ocupado

```
int rehash (int x){  
    return ++x % 7;  
}
```

Vamos inserir os números:

5, 7, 14, 21, 19 e 1

Impossível inserir o 21!!!

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

ocupado

0	7
1	14
2	
3	
4	
5	5
6	

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	

livre

```
int rehash (int x){  
    return ++x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	19

```
int rehash (int x){  
    return ++x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	19

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	
3	
4	
5	5
6	19

ocupado

```
int hash (int x){  
    return x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

livre

0	7
1	14
2	
3	
4	
5	5
6	19

```
int rehash (int x){  
    return ++x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com Rehash

- Suponha uma tabela de tamanho 7

0	7
1	14
2	1
3	
4	
5	5
6	19

```
int rehash (int x){  
    return ++x % 7;  
}
```

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Direta com *Rehash*

- Suponha uma tabela de tamanho 7

0	7
1	14
2	1
3	
4	
5	5
6	19

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Exercício Resolvido (3)

- Suponha uma tabela de tamanho 7

0	7
1	14
2	1
3	
4	
5	5
6	19

Exercício: Quantas comparações são necessárias para pesquisar o:

- a) 5 b) 21
c) 19 d) 6

Exercício Resolvido (3)

- Suponha uma tabela de tamanho 7

0	7
1	
2	
3	
4	
5	
6	19

Pause!

a) 5

c) 19

as comparações
para pesquisar o:

b) 21

d) 6

Exercício Resolvido (3)

- Suponha uma tabela de tamanho 7

0	7
1	14
2	1
3	
4	
5	5
6	19

Exercício: Quantas comparações são necessárias para pesquisar o:

- a) 5 1 cmp b) 21
c) 19 d) 6

Exercício Resolvido (3)

- Suponha uma tabela de tamanho 7

0	7
1	14
2	1
3	
4	
5	5
6	19

Exercício: Quantas comparações são necessárias para pesquisar o:

a) 5

b) 21 2 cmp

c) 19

d) 6

Exercício Resolvido (3)

- Suponha uma tabela de tamanho 7

0	7
1	14
2	1
3	
4	
5	5
6	19

Exercício: Quantas comparações são necessárias para pesquisar o:

- a) 5 b) 21
c) 19 2 cmp d) 6

Exercício Resolvido (3)

- Suponha uma tabela de tamanho 7

0	7
1	14
2	1
3	
4	
5	5
6	19

Exercício: Quantas comparações são necessárias para pesquisar o:

a) 5

b) 21

c) 19

d) 6

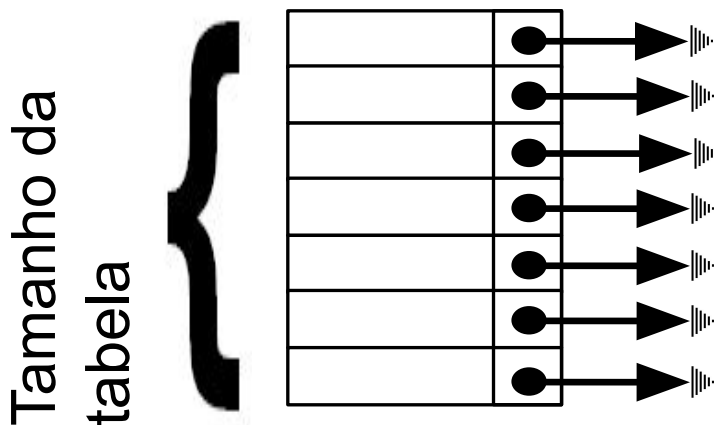
2 cmp

Gerenciamento de Colisões

- *Hash* direta com área de reserva (*overflow*)
- *Hash* direta com *rehash*
- ***Hash* indireta com lista flexível simples**

Hash Indireta com Lista Flexível Simples

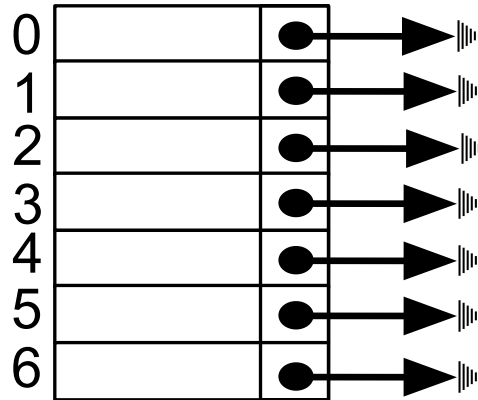
- Suponha uma tabela de tamanho 7



```
int hash (int x){  
    return x % tamTabela;  
}
```

Hash Indireta com Lista Flexível Simples

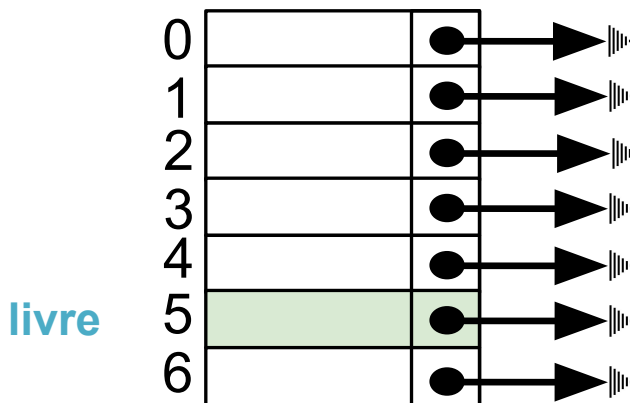
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

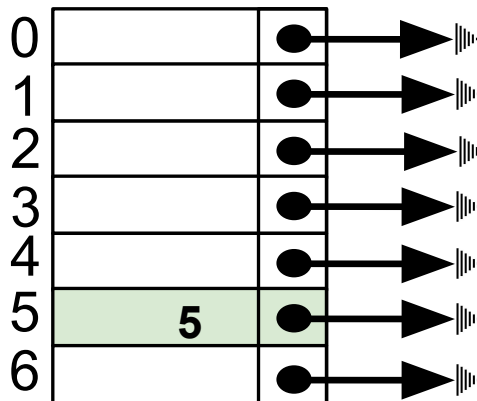
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

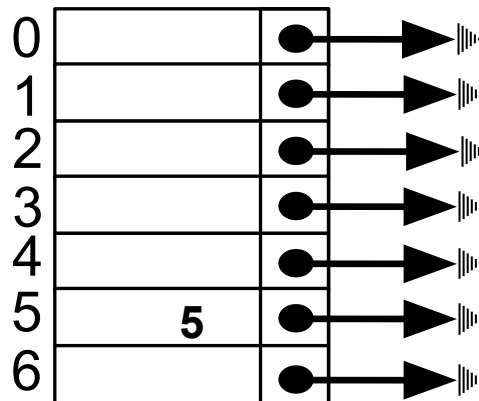
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

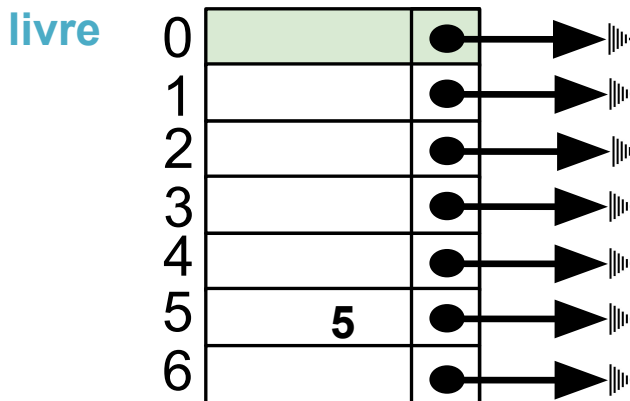
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

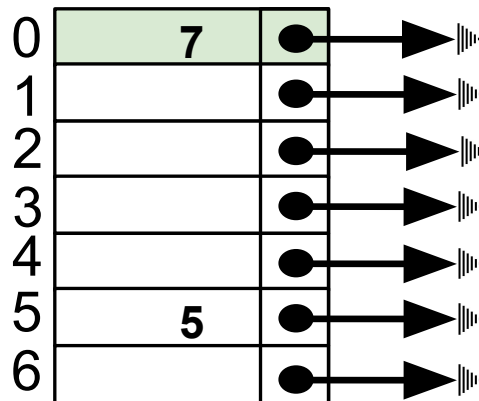
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

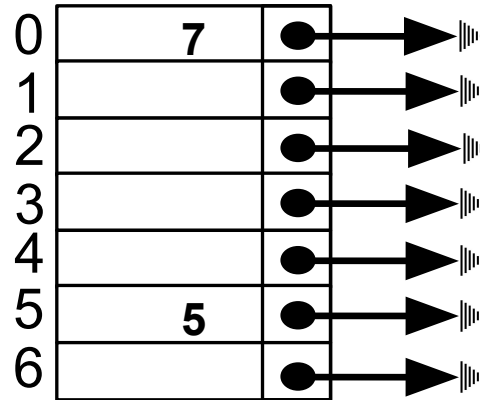
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

- Suponha uma tabela de tamanho 7

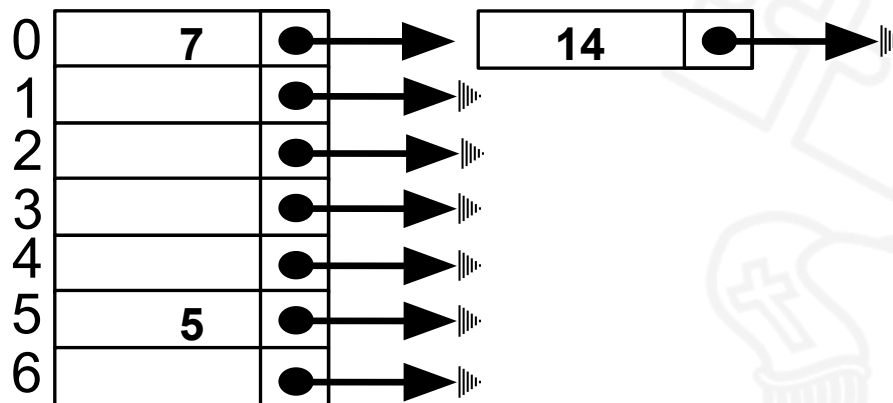
ocupado

0	7	●	→
1		●	→
2		●	→
3		●	→
4		●	→
5	5	●	→
6		●	→

Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

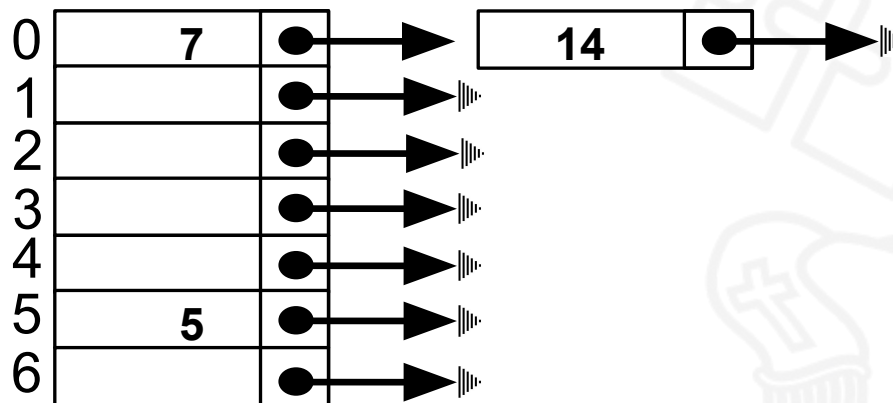
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

- Suponha uma tabela de tamanho 7

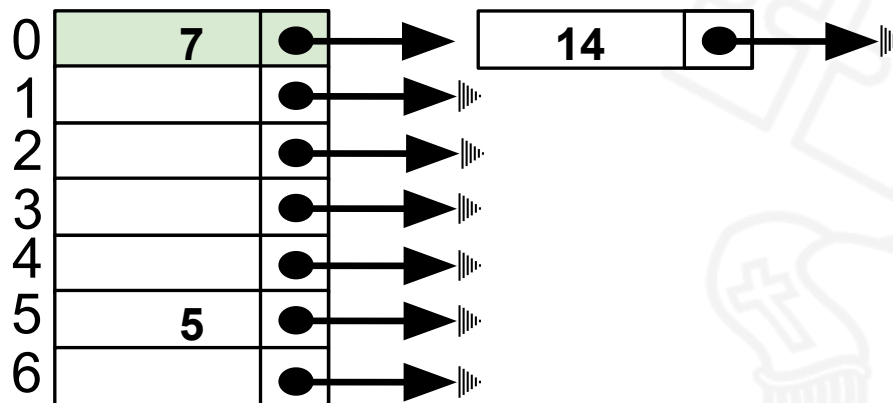


Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

- Suponha uma tabela de tamanho 7

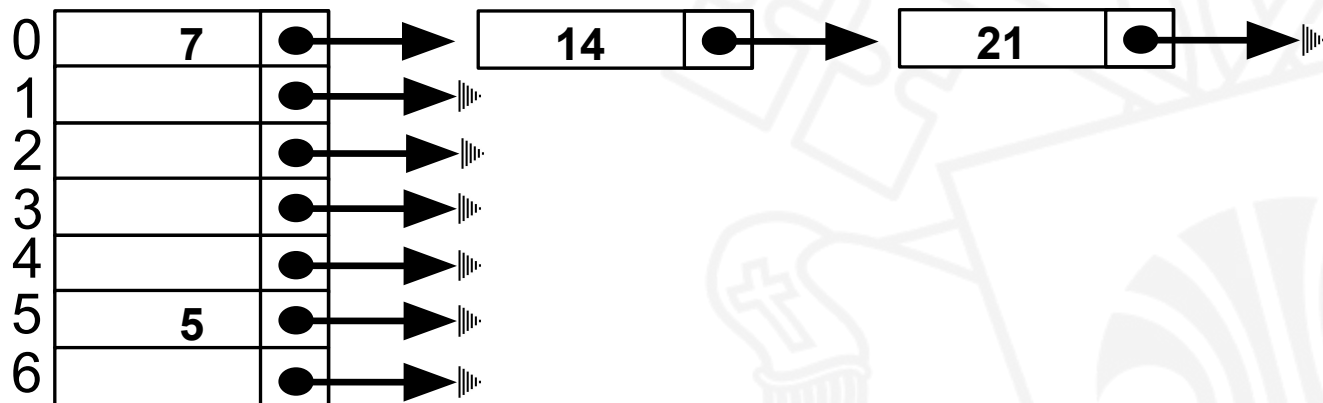
ocupado



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

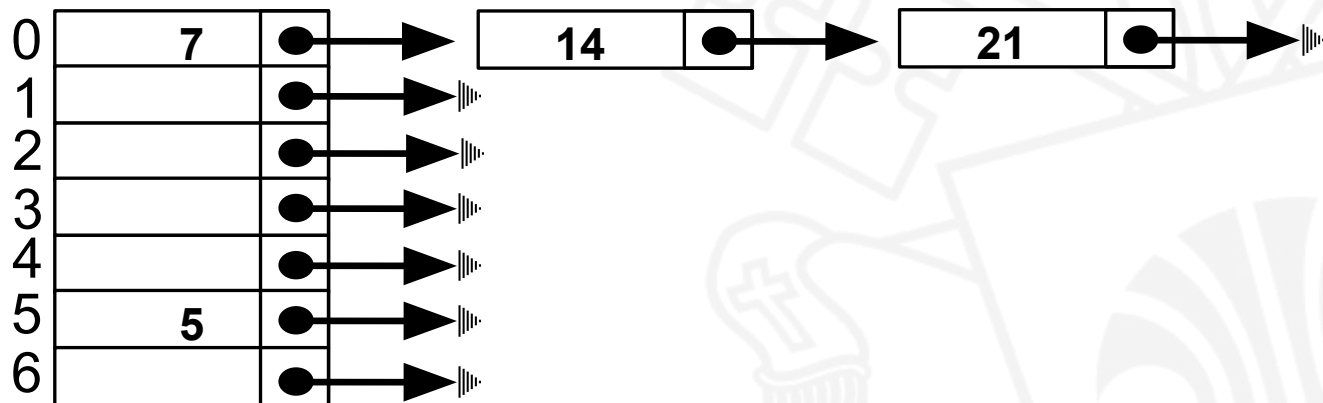
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

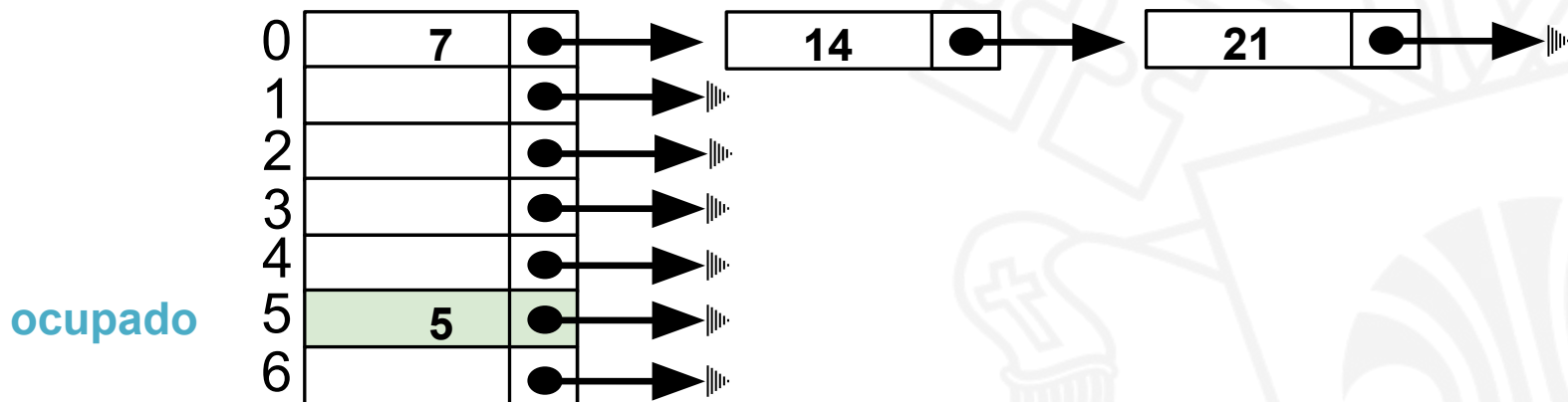
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

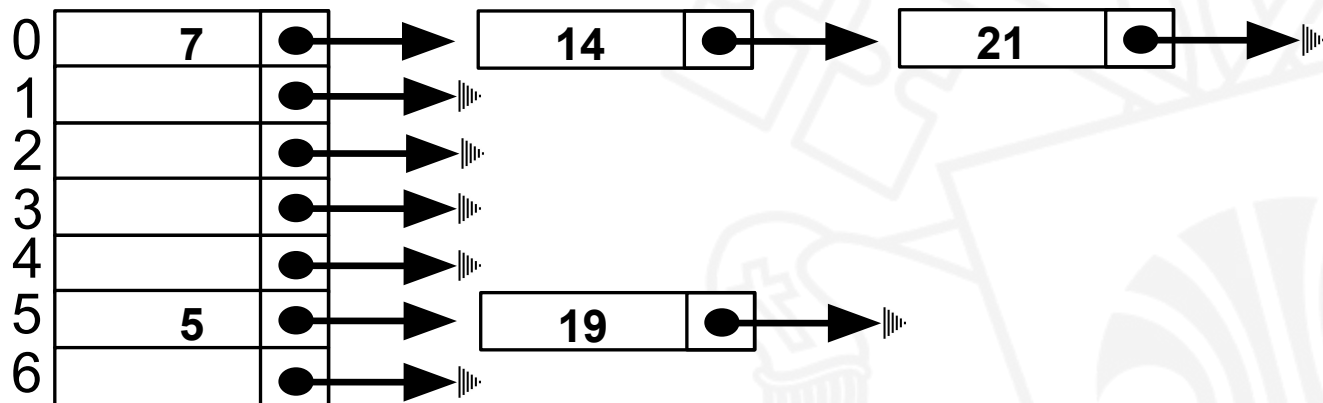
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

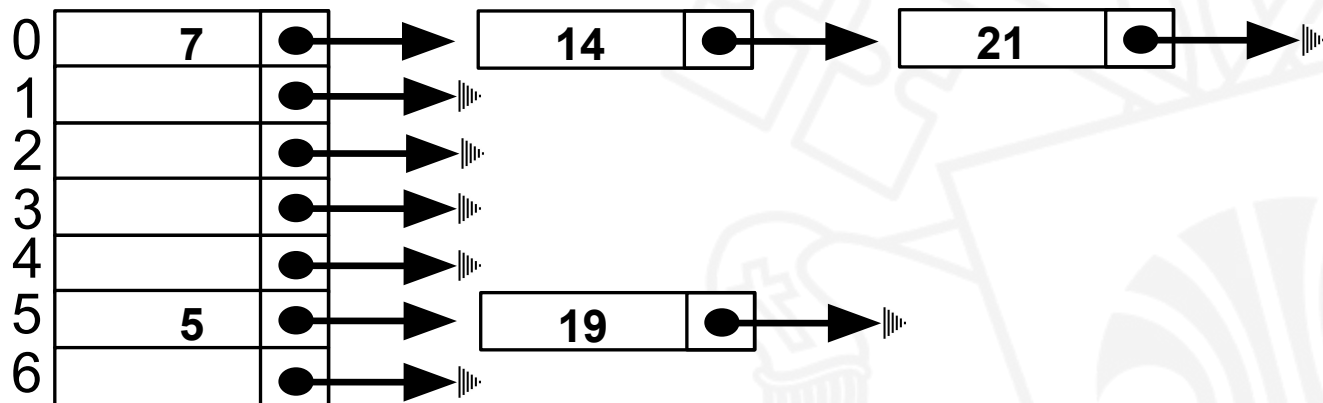
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, **19** e 1

Hash Indireta com Lista Flexível Simples

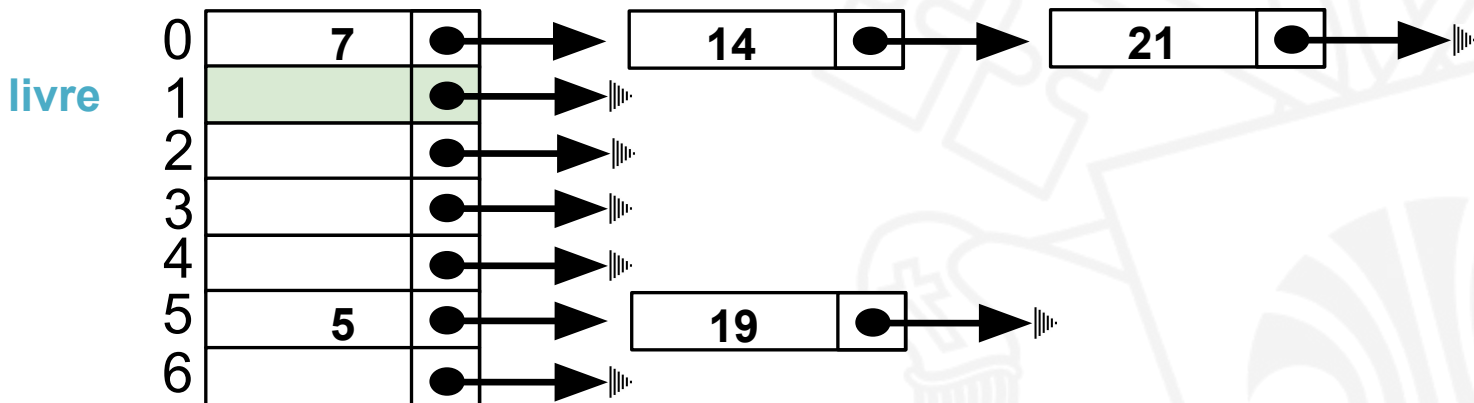
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

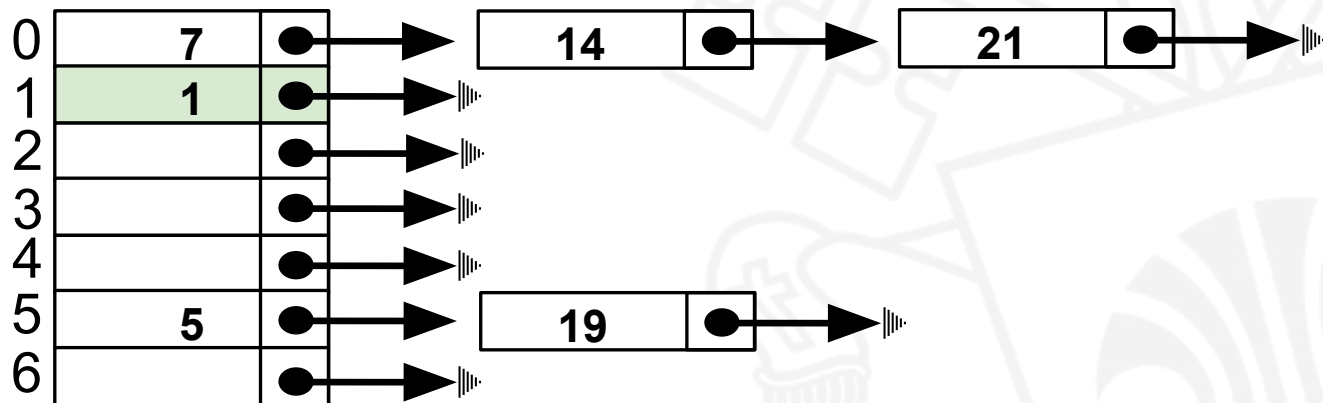
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

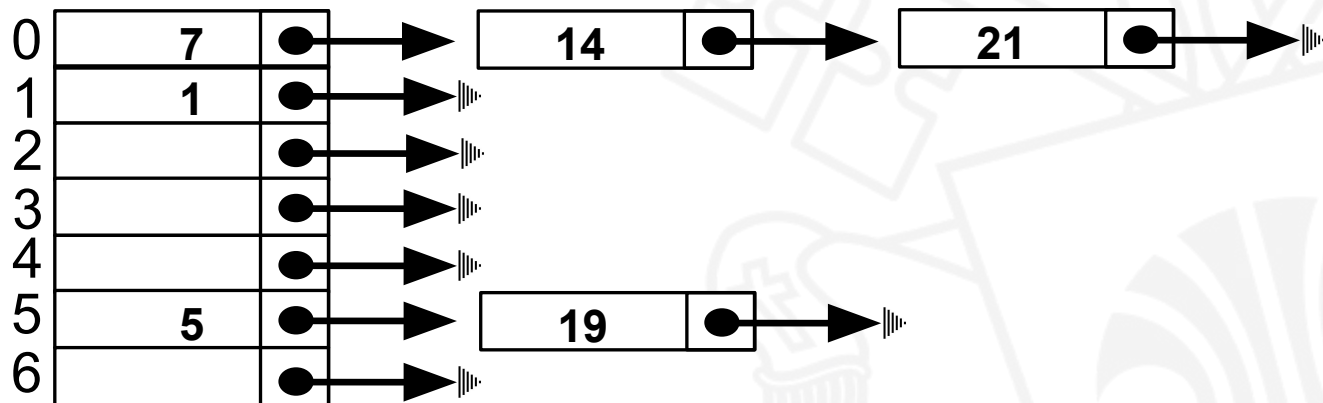
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Hash Indireta com Lista Flexível Simples

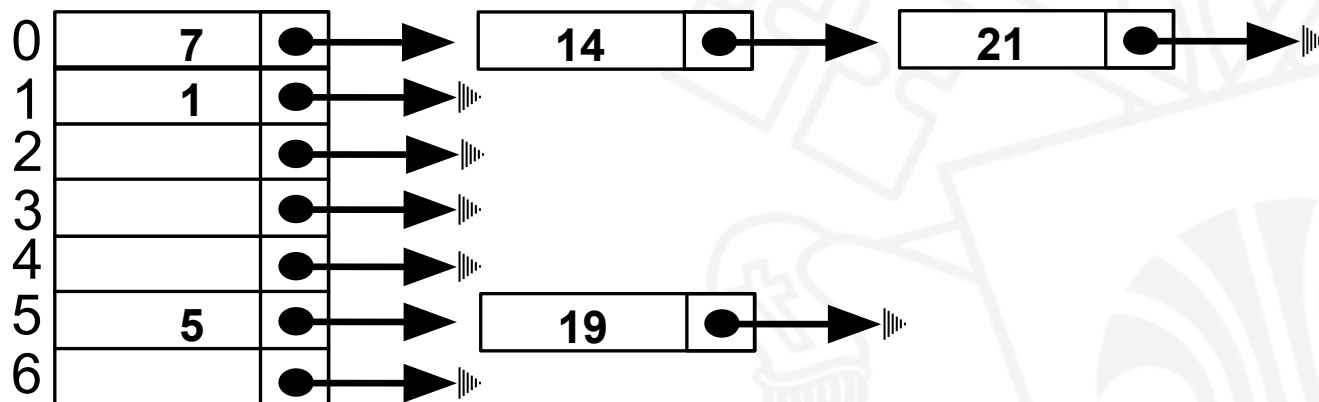
- Suponha uma tabela de tamanho 7



Vamos inserir os números:
5, 7, 14, 21, 19 e 1

Exercício Resolvido (4)

- Suponha uma tabela de tamanho 7



Quantas comparações são necessárias para pesquisar o:

a) 5

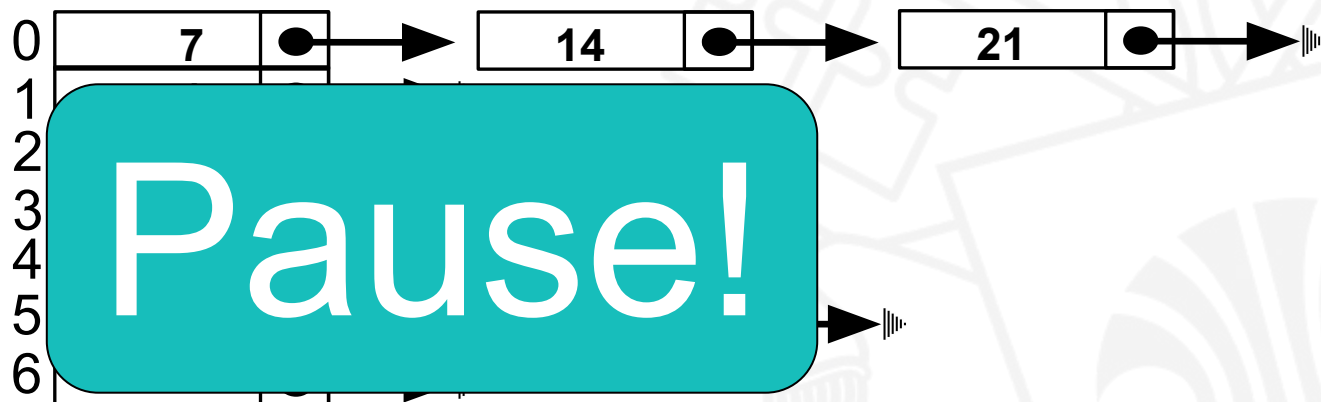
b) 21

c) 19

d) 6

Exercício Resolvido (4)

- Suponha uma tabela de tamanho 7

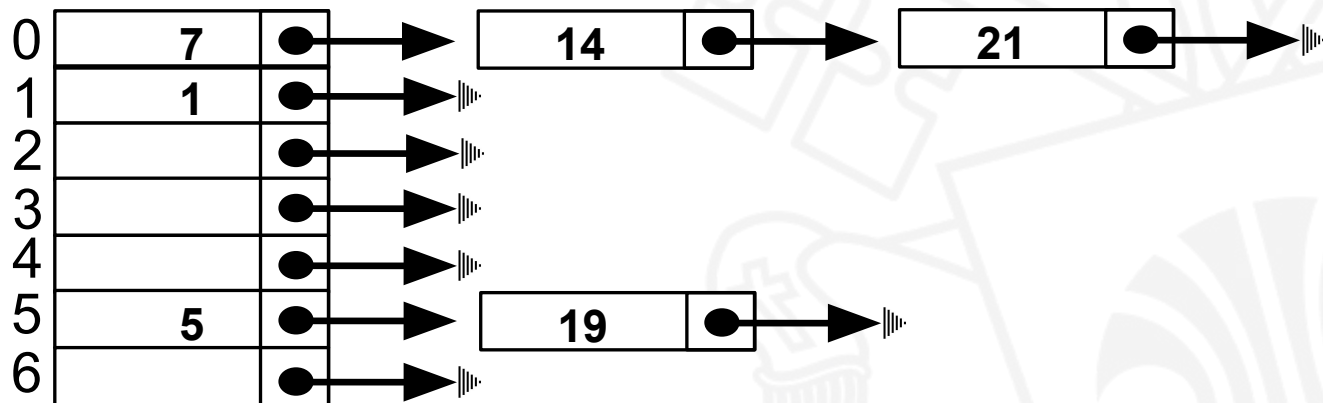


Quantas comparações são necessárias para pesquisar o:

- a) 5 b) 21 c) 19 d) 6

Exercício Resolvido (4)

- Suponha uma tabela de tamanho 7

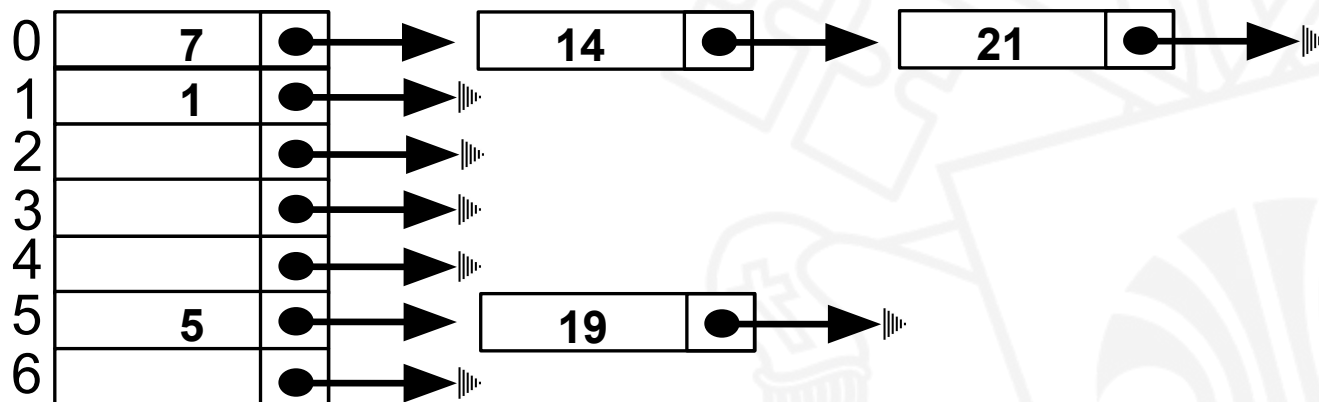


Quantas comparações são necessárias para pesquisar o:

- a) 5 1 cmp b) 21 c) 19 d) 6

Exercício Resolvido (4)

- Suponha uma tabela de tamanho 7



Quantas comparações são necessárias para pesquisar o:

a) 5

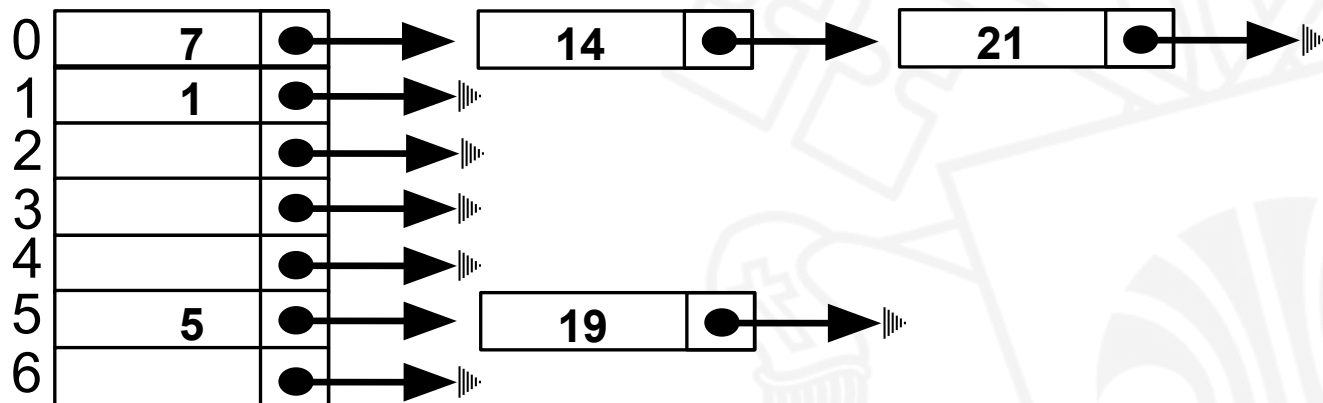
b) 21 3 cmp

c) 19

d) 6

Exercício Resolvido (4)

- Suponha uma tabela de tamanho 7



Quantas comparações são necessárias para pesquisar o:

a) 5

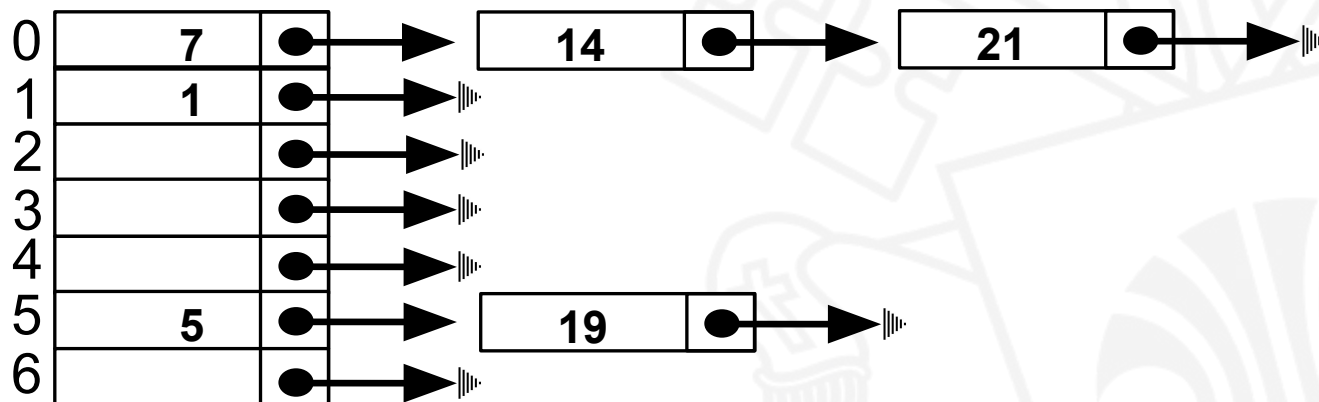
b) 21

c) 19 2 cmp

d) 6

Exercício Resolvido (4)

- Suponha uma tabela de tamanho 7



Quantas comparações são necessárias para pesquisar o:

- a) 5 b) 21 c) 19 d) 6 **1 cmp**